Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

EcoRouting Engine: Hazard-Aware Path Optimization and Fill-Rate-Adaptive Waste Scheduling

Student/Team Information

Team Name: *Team #* *PBL Mentor:*	*EcoGraph* *DSCPP-III-2026-T186* *Mr. Navneet Rajput*
Team member 1 (Team Lead)	*Jain, Tejas* *Student ID: 2510011541* *Roll No. 2028228, Sec A* *aispace101@gmail.com* 
Team member 2	*Negi, Apoorv* *Student ID: 2510011088* *Roll No. 2027640, Sec D* *apoorvnegi17@gmail.com* 
Team member 3	*Nigam, Yash* *Student ID: 2510010441* *Roll No. 2028289, Sec H* *nigamyash07@gmail.com* 

PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

Watch a municipal garbage truck for a few days and you notice something odd: it takes the same streets in the same order every day, whether or not there is anything to pick up. The route comes from habit, not from the shortest path between the depot, the homes and the disposal sites, and that wastes fuel, driver hours and public money. Two more problems make it worse. Hazardous waste such as medical sharps, old chemicals and batteries is often picked up just like ordinary household waste, when it should be collected quickly and kept away from homes and markets on its way to disposal. And every disposal site is treated as if it fills at the same rate, so a fixed weekly schedule either visits one that is almost empty or misses one that is overflowing. We wanted a system that sees the city for what it really is, a graph, and plans collection around that.

EcoGraph is our answer. A C++ backend stores the city as a weighted graph in which every road is tagged with its zone, and it answers three questions whenever asked: what is the shortest route to collect from a set of homes, which homes actually need a visit today based on how fast they fill, and how to get hazardous waste to disposal while staying away from residential streets. All of this is available through a REST API, and a PixiJS front end shows the trucks driving their routes live. The system is online at `ecograph-xl8q.onrender.com`.

Updated Project Approach and Architecture

Design. Everything runs in one C++17 program that serves both the REST API and the web page on the same port. `controllers.cpp` deals with HTTP and JSON and keeps shared data behind a mutex. The real logic lives in separate classes (`Graph`, Dijkstra, `Scheduler`, the waste classifier, the route builder and `Simulation`), which all work on an in-memory copy of the city, loaded from `city.json` and stored as an adjacency list.

Routing. `RouteStrategy` is an abstract base class. Its Priority subclass decides the stop order and always goes to the fullest home first.

Protocol. The front end talks to the server over plain REST with JSON: 15 endpoints.

Front end. Plain JavaScript. Alongside the map it has a day-by-day simulation with rules to simulate a limited number of trucks.

Libraries. cpp-httplib for the server, nlohmann/json for JSON and PixiJS for drawing the map.

Deployment. The Docker image is built in two stages: the first compiles the server, and the second keeps only the binary, `city.json` and the front-end files, so the final image stays small. It runs as a single container on Render, and the page and the API share one address.

System overview (diagrams (a)–(h) on the next pages)

1. Each simulated day starts by moving the clock forward one day.

2. A home becomes due once the days since its last pickup reach capacity / fill rate (between 1 and 14 days). The fullest home goes first (d).

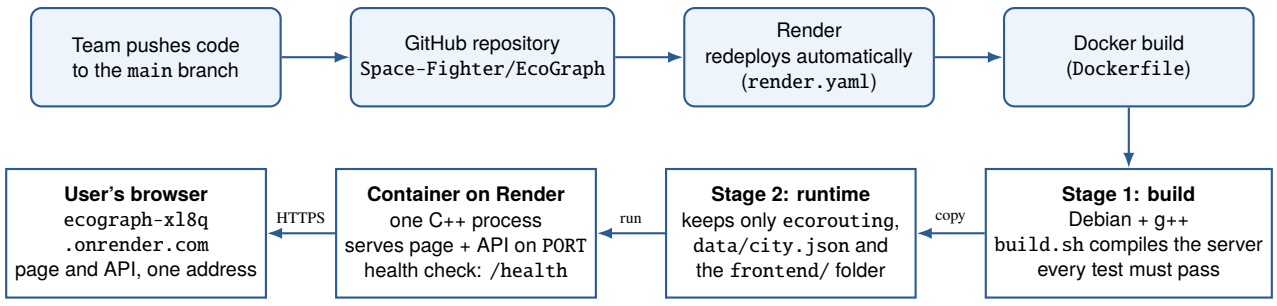
3. Due disposal sites are emptied, hazardous homes are handed to the hazard trucks (g), and the remaining homes are sorted and loaded onto trucks before routing (f).

4. Each truck then joins shortest paths in the order of its list:

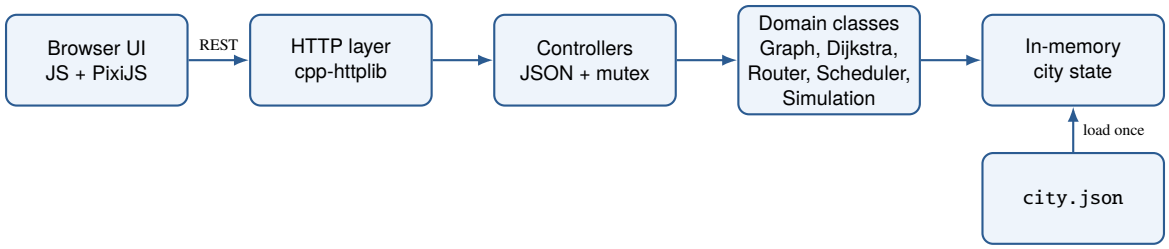
```
current = DEPOT;  visited = { }
for each home in order:
  skip it if in visited
  path = Dijkstra(current → home);  if none, mark unreachable
  empty later stops on the path, add them to visited          (en-route pickup, (e))
  empty home, add to visited;  current = home
then visit the nearest disposal site for each waste type collected
```

5. Every served home is marked as collected today. Homes the truck can't reach are reported as failed, and homes that didn't fit on a truck move up the list for the next day.

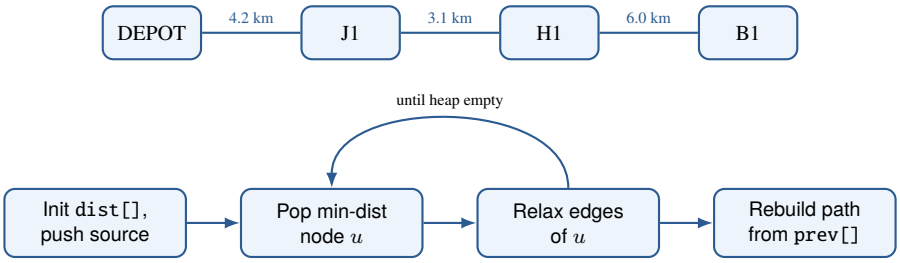
Deployment Setup



(a) System Pipeline



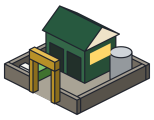
(b) Graph Storage and Dijkstra



(c) Types

Every icon below is the one the map uses for that type.

Depot



Depot

The depot is the base for the whole fleet. Normal and hazard trucks both leave from here each day.

Homes (collection points)

Each point on the map stands for a small cluster of buildings, drawn by the zone it sits in.



Residential: houses



Commercial: shops and offices



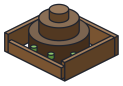
Industrial: factories and warehouses



Hazardous sites: clinics, labs and workshops, marked with a warning sign

Waste types and their disposal sites

The classifier reads each home's waste description and matches keywords. Each waste type goes to its own kind of disposal site.

Disposal site	Waste type	Example keywords	Hazardous?
	General	Anything that matches no other type	No
	Recyclable	plastic, bottle, can, glass, paper, cardboard, newspaper	No
	Compost	food, organic, vegetable, fruit, leaves, peel, garden	No
	Medical	syringe, bandage, needle, blood, gauze, surgical	**Yes**
	Chemical	solvent, acid, battery, paint, pesticide, bleach	**Yes**

Trucks



Normal truck

Collects general, recyclable and compost waste from homes, shops and factories.



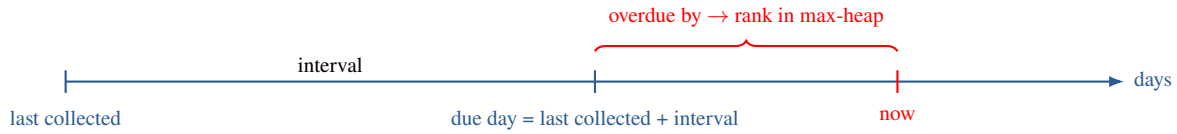
Hazard truck

Collects medical and chemical waste and takes it to a medical or chemical disposal site.

Road types

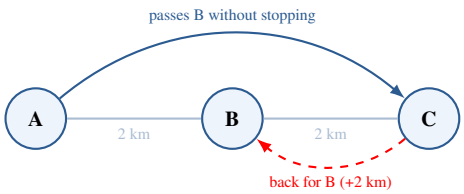
- Arterial: main roads between localities
- Commercial: roads through markets
- Industrial: roads in industrial belts
- Residential: roads through neighbourhoods

(d) Fill-Rate Scheduling

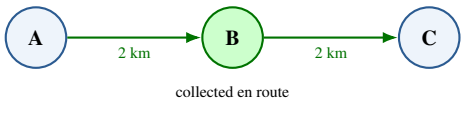


(e) En-Route Pickup

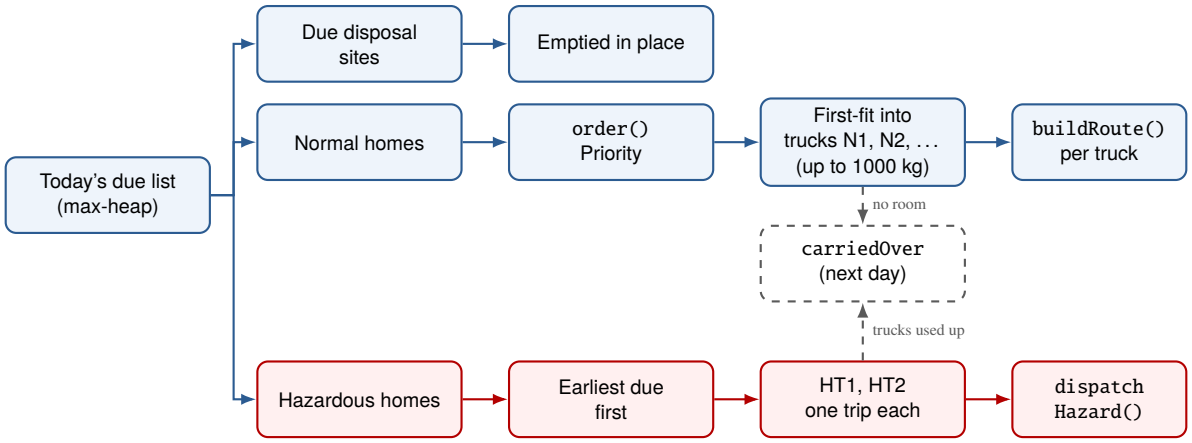
Literal order A → C → B: 6 km



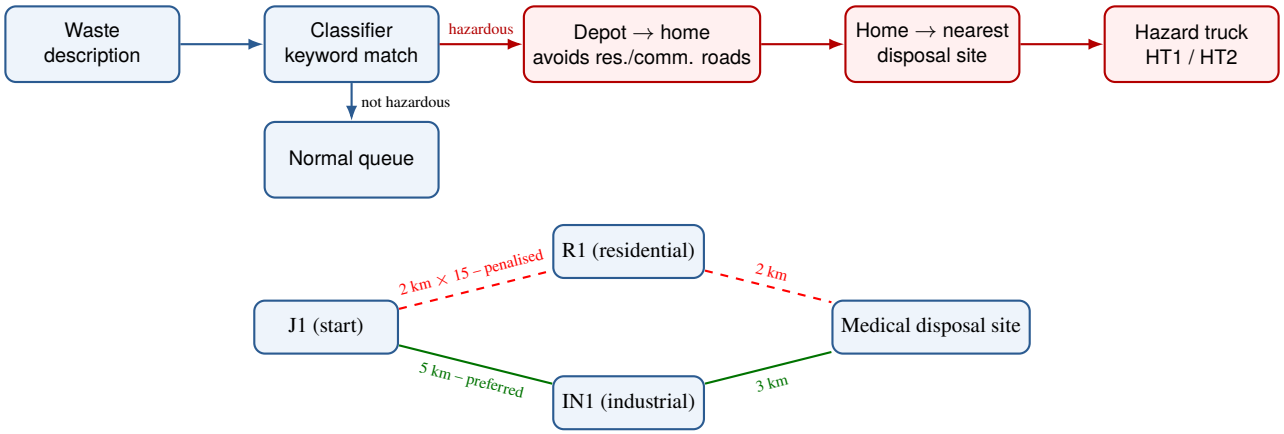
EcoGraph: A → B → C: 4 km



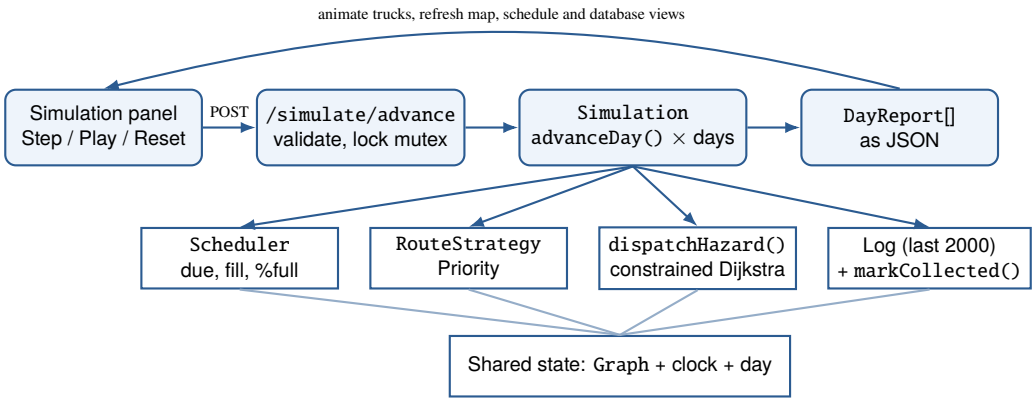
(f) Truck Fleet and Capacity



(g) Automatic Hazard Dispatch and Risk-Weighted Dijkstra



(h) Simulation Architecture



Tasks Completed

Task Completed	Team Member
Built the city as a weighted graph (adjacency list) with a zone on every road, and a loader that reads the Dehradun map (70 nodes, 107 roads) from JSON	Tejas Jain (Lead)
Set up the cpp-httplib server that serves the web page and 15 REST endpoints, with a common error format and a mutex around shared data	Tejas Jain
Wrote Dijkstra's algorithm and a zone-aware version, both able to rebuild the path	Tejas Jain
Wrote the priority route builder (fullest home first), with en-route pickup and unloading at the nearest disposal site	Tejas Jain
Wrote the keyword-based waste classifier and the automatic hazard dispatch to the nearest medical or chemical disposal site, which keeps off residential and commercial roads wherever another road exists	Tejas Jain
Wrote the fill-rate scheduler, which keeps due homes in a max-heap	Tejas Jain
Built the day-by-day simulation: limited trucks, truck capacity, carry-over to the next day and a collection log	Tejas Jain, Apoorv Negi, Yash Nigam
Wrote unit tests for the backend and a check for the city data	Yash Nigam, Apoorv Negi, Tejas Jain
Wrote the README: build steps, folder layout and API reference	Apoorv Negi, Yash Nigam
Built the PixiJS map with truck animation, hazard routes and a legend	Yash Nigam, Tejas Jain, Apoorv Negi
Built the schedule, simulation and database views and the light/dark theme	Tejas Jain, Yash Nigam
Wrote the multi-stage Dockerfile and `.dockerignore`	Yash Nigam, Tejas Jain
Deployed the app on Render as a single container, with a health check on `/health` and automatic redeploys from `main`	Tejas Jain
Set `PORT`, `CITY_FILE` and `FRONTEND_DIR` in the image, and served the page from the same address as the API (`ECO_API_BASE` left empty)	Tejas Jain, Yash Nigam
Added a one-line `docker run` command to the README	Yash Nigam
Added the live demo link to the README	Tejas Jain
Improved the front end	Tejas Jain
Cleaned up the code	Tejas Jain, Apoorv Negi
Wrote the Phase-I, Phase-II and Phase-III reports and presentations	Tejas Jain, Apoorv Negi, Yash Nigam

Challenges/Roadblocks

Road classes beyond residential and commercial. Our first map had only residential and commercial roads, which left the hazard truck with no safer road to choose. Real cities also have main roads linking localities, and industrial belts where disposal sites usually are, so we added *arterial* and *industrial* road types. When planning a hazard route, a residential road counts as 15 times its length and a commercial road as 10 times, so the truck sticks to main and industrial roads and enters a neighbourhood or market only for the stretch it can't avoid.

Banning residential and commercial roads completely. Our first hazard router simply refused to drive on any residential or commercial road. That only worked because we had given every hazardous site an extra main-road connection in the city file. A real clinic in a market or a lab in a colony can only be reached through its own local street, and with a full ban the truck had no way to get there, so the pickup failed. We replaced the ban with a weighted version of Dijkstra: each road's length is multiplied by a risk factor (industrial 1×, arterial 1.5×, commercial 10×, residential 15×). Populated streets are no longer forbidden, just very expensive, so the truck uses one only for the last stretch it can't avoid. Each hazard trip now also reports how many km it drove on populated roads (see the figure below).

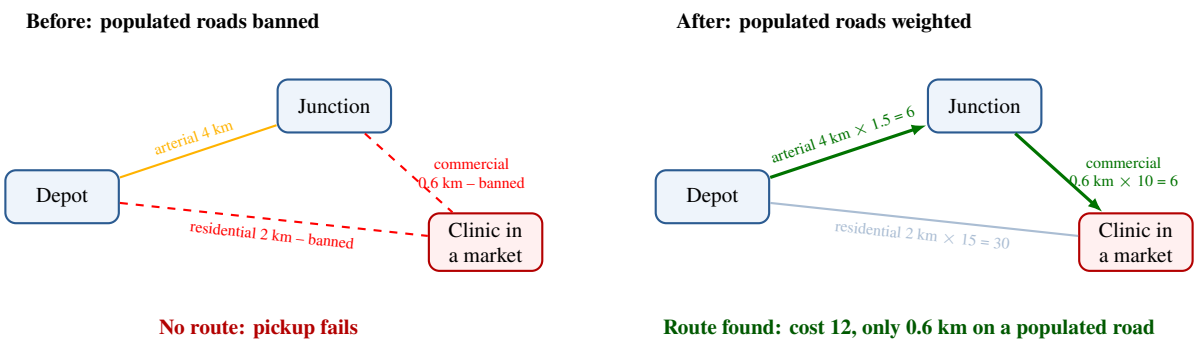
Trucks driving past houses on their own list. At first the router followed the scheduler's order exactly. If the list was A, C, B and B sat on the road from A to C, the truck drove straight past B, reached C, then came back for B. We fixed this with en-route pickup: for each leg we check whether a later stop lies on the path, collect it on the way, and skip it when the list gets there.

Scheduling that never showed its effect. With unlimited trucks every due home was served every day, nothing ever overflowed, and the priority order only changed the order of stops, never who got served. So we limited each truck to 1000 kg and pushed whatever didn't fit to the next day. One catch: an overflowing home heavier than a whole truck would never fit, so an empty truck always takes its first home.

Shared state across requests. cpp-httplib handles requests on several threads at once, and routing, collection and the simulation all change the same graph. We put all shared state behind one mutex so two requests can't step on each other.

Compiler on Windows. The old MinGW g++ couldn't build cpp-httplib because it lacks proper thread support, so we switched to GCC 13.

Hazard routing: full ban vs. weighted roads



Project Outcome/Deliverables

- A C++17 server that loads a city and answers routing, hazard dispatch, classification, scheduling and simulation requests through a REST/JSON API.
- A web page that draws the Dehradun map, animates every truck's route, highlights hazardous trips and lets you move the city forward one day at a time.
- Unit tests for every core module, and a sample city file that anyone can edit.
- A live demo at `ecograph-xl8q.onrender.com`, the README and API docs, and the Phase-II and Phase-III reports and presentations.

In the end, what matters to us is simple. For any list of homes, the system should find a shorter route than a fixed one, keep hazardous waste off residential and commercial streets wherever another road exists, and stop sending trucks to disposal sites that are nowhere near full.

Progress Overview

The project is now 100% complete. At Phase-II we were at about 80%, with three things left: deployment, more work on the front end, and some code cleanup. All three are done. The graph model, both versions of Dijkstra, the route builder, the waste classifier, the fill-rate scheduler, the simulation, the REST API and the front end all work, and the same system is running online.

Nothing fell behind. In Phase-I we planned to deploy in Phase-III, and that is when it happened. The day-by-day simulation and the limited truck fleet were extras: we added them because a single route on a static map couldn't show whether our scheduling really worked.

The live link is in the README, and we are ready for the Phase-III presentation and the final demo.

Codebase Information

Repository: `github.com/Space-Fighter/EcoGraph` **Branch:** `main`

Hosted demo: `https://ecograph-xl8q.onrender.com`

`server/` holds the C++ code, headers, tests and `data/city.json`; `frontend/` holds the HTML, CSS and JavaScript modules.

Key files

File	What it does
`server/src/main.cpp`	Starts the server; reads the port, city file and front-end folder
`server/src/controllers.cpp`	The 15 REST endpoints: parses JSON, calls the domain classes, guards shared state with a mutex
`server/src/graph.cpp`	City graph: nodes and zone-tagged roads stored as an adjacency list
`server/src/dijkstra.cpp`	Plain and zone-constrained shortest path
`server/src/scheduler.cpp`	Collection interval, due, overdue and % full; max-heap of due locations
`server/src/waste_classifier.cpp`	Keyword rules: medical, chemical, recyclable, compost (else general)
`server/src/router.cpp`	Priority route builder, en-route pickup, nearest disposal site unloading, hazard dispatch
`server/src/simulation.cpp`	Advances days, loads trucks, runs routes and hazard trips, keeps the collection log
`server/src/city_loader.cpp`	Reads `city.json` into the graph
`server/data/city.json`	The Dehradun city: localities, nodes, roads, zones, fill rates
`server/tests/`	Unit test programs (Dijkstra, domain, simulation, generator, city data)
`frontend/js/graph-view.js`	PixiJS map and truck route animation
`frontend/js/schedule-view.js`, `simulation-view.js`, `database-view.js`	Schedule, simulation and database panels
`frontend/js/api.js`	Thin `fetch` wrapper around the REST API
`Dockerfile`, `.dockerignore`, `render.yaml`	Container image and Render deployment settings

Important commits and the modules they added

Commit	Modules / checkpoint
Graph model and Dijkstra	`graph`, `dijkstra` (plain and constrained), Dijkstra tests
REST API server, city data and front end	`scheduler`, `waste_classifier`, `router` (Priority), `controllers`, map view
Day-by-day simulation and database view	`simulation`, database view, simulation tests
Hazard truck dispatch and limited fleet	automatic hazard trucks, truck count and capacity in `simulation` and `router`
City mapped to Dehradun (km/kg units)	real coordinates and localities
Full README with API reference	README, API docs
Schematic locality layout	cleaner map layout of the localities
En-route pickup	en-route collection in `router`
Add Dockerfile (`5ab2551`)	Docker image and `docker run` instructions
Add Render deployment config (`0b8130f`)	`render.yaml`, updates to `Dockerfile` and `.dockerignore`

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Unit: Dijkstra (`test_dijkstra`)	Pass	Results match graphs we solved by hand, and an unreachable target is handled cleanly
Unit: domain (`test_domain`)	Pass	Classifier rules, scheduler intervals kept between 1 and 14 days, priority order and tie-breaks, the zone-aware route avoids a blocked road, hazard split
Unit: simulation (`test_simulation`)	Pass	Trucks never go over capacity, leftover homes carry over to the next day, hazard trucks work
Unit: generator (`test_generator`)	Pass	New nodes are unique, connected to the roads, reachable and placed inside their locality
Data: city file (`test_city_data`)	Pass	The Dehradun city file has its depot, localities and hazard homes set up correctly
API checks on the hosted site (curl)	Pass	`/health`, `/graph`, `/schedule`, `/classify`, `/route` and `/simulate/advance` all return the expected JSON, and bad JSON gets a clean 400 error
Browser check on the hosted site	Pass	The map, truck animation and database view all load and run

Deliverables Progress

- C++ backend: routing, hazard dispatch, classifier, scheduler, simulation, REST API – **Completed**
- Browser front end with map, route animation, schedule and database views – **Completed**
- Dehradun city data – **Completed**
- Unit tests (4 suites + city data check) – **Completed**
- README and API documentation – **Completed**, including the live link
- Hosted demo (Dockerfile + deployment on Render) – **Completed**
- Phase-II report and presentation – **Completed**
- Phase-III report – **Completed**; Phase-III presentation and final demo – **Pending**